CDWIA v2 — Tool & AWS Service Comparison Table

Step-by-step, in request-flow order: what industry-standard tool would build this, what AWS service replaces it, why each was chosen, and what it wins over the alternatives

---

How to read this table

Sequence follows the request path from the architecture diagrams: client → edge → control plane → data plane → synthesis → shared services → ingestion. For each step: the tool a team would typically reach for building this themselves, the AWS-managed equivalent, the reasoning for picking one over the other, and the concrete advantage it has over other options in its category (not just "vs building it yourself").

---

1. Client / presentation layer

#	Component	Industry tool	AWS service	Why chosen	Advantage over alternatives
1	Web/chat UI	Streamlit, React, custom Teams bot	AWS Amplify (React hosting) + CloudFront	Amplify gives CI/CD, hosting, and auth hooks in one managed flow, so the frontend ships without a separate deploy pipeline	Over self-hosted React (Vercel/Netlify-style): native IAM/Cognito integration means no separate auth bridge to build. Over Streamlit for production: Streamlit is fast for internal prototypes but isn't built for multi-tenant, scaled consumer-facing traffic — Amplify is

---

2. Edge — identity, protection, entry point

#	Component	Industry tool	AWS service	Why chosen	Advantage over alternatives
2	CDN / edge cache	Cloudflare, Fastly, Akamai	Amazon CloudFront	Native integration with WAF and API Gateway in the same account/IAM boundary — one less	Over Cloudflare: no separate billing/API surface to integrate; origin shielding and Lambda@Edge let

#	Component	Industry tool	AWS service	Why chosen	Advantage over alternatives
				cross-vendor trust relationship to manage	you run logic at the edge without leaving the AWS network
3	Web Application Firewall	ModSecurity, ownCloud WAF, ex-Imperva	AWS WAF	Managed rule groups (SQLi, common exploits) updated by AWS, attachable directly to CloudFront/API Gateway with no extra infra	Over self-managed ModSecurity: no patching/rule-maintenance burden; scales automatically with traffic, no separate WAF cluster to run
4	Identity provider / OIDC	Okta, Auth0, Azure AD	Amazon Cognito (or Cognito federating to Azure AD/Okta)	If already on Azure AD/Okta, Cognito can federate rather than replace it — gives you a single token-issuing boundary API Gateway trusts natively	Over running Okta/Auth0 standalone: no separate JWKS endpoint to manage trust with; native API Gateway Cognito authorizer means zero custom token-verification code
5	API gateway	Kong, Apigee, Tyk	Amazon API Gateway	Built-in rate limiting, request validation, and native Lambda/Step Functions integration — the exact three things the gateway layer needs (Section 2 of the design doc)	Over Kong/Apigee: no separate cluster to run and patch; usage-based pricing avoids paying for idle capacity at low traffic, which self-hosted gateways still incur

3. Control plane — decisions before execution

#	Component	Industry tool	AWS service	Why chosen	Advantage over alternatives
6	Deterministic	scikit-learn,	AWS Lambda	Lambda keeps the	Over running

#	Component	Industry tool	AWS service	Why chosen	Advantage over alternatives
	classifier	spaCy, a fine-tuned small transformer	running the model, or SageMaker Serverless Inference for a trained model	classifier as cheap, stateless, millisecond-latency compute — matching the design doc's requirement that this stay off the LLM cost path entirely	the classifier on a managed always-on endpoint: Lambda's pay-per-invocation model matches the classifier's bursty, sub-50ms workload exactly — you're not paying for idle GPU/CPU between requests
7	LLM fallback planner	LangChain's planner abstraction, custom prompt orchestration	Amazon Bedrock (Claude via `InvokeModel`)	Only invoked on low classifier confidence — Bedrock gives pay-per-token access to Claude without hosting or provisioning any model infrastructure	Over self-hosting an open model (e.g. Llama on SageMaker): no GPU capacity planning, and Bedrock's per-token billing means idle time between fallback invocations costs nothing
8	Policy engine (RBAC/ABAC)	Open Policy Agent (OPA)	Amazon Verified Permissions	Verified Permissions is a managed Cedar-policy engine — same "declarative, versioned, testable" property the design doc calls for in OPA, without running and scaling an OPA server yourself	Over self-hosted OPA: no server to patch, scale, or keep highly available; Cedar policies get native IAM-style audit trails

#	Component	Industry tool	AWS service	Why chosen	Advantage over alternatives
					in CloudTrail, which OPA's Rego doesn't give you out of the box
9	Cost-tiered model router	LiteLLM, a custom router with a provider-abstraction layer	Bedrock's built-in cross-region inference + a thin Lambda wrapper implementing the fallback-chain logic	Bedrock already exposes multiple models (Haiku/Sonnet/Opus-equivalent tiers) behind one API and one billing relationship, so the router only needs to choose a model ID, not manage multiple provider SDKs/credentials	Over LiteLLM with multiple external providers: one contract, one set of credentials, one compliance boundary (data residency, logging) instead of juggling several vendors' terms of service
10	Agent orchestration (fan-out, retries, state)	LangGraph, CrewAI, AutoGen	AWS Step Functions	Step Functions gives explicit state machines with native parallel/Map states, built-in retry policies, and a visual execution history for debugging — the same "explicit state machine, not a plain chain" requirement the design doc calls out for LangGraph	Over LangGraph: no Python process managing state in memory that needs its own scaling/failure story — Step Functions' state is durable and managed by AWS, and retries/backoff are declarative config, not code you maintain

4. Data plane — SQL path

#	Component	Industry tool	AWS service	Why chosen	Advantage over alternatives
11	SQL validation (AST parsing)	sqlglot (Python library)	sqlglot, run inside Lambda — **no AWS-native replacement exists or should exist here**	This stays as an application-level library regardless of cloud provider — the value is in the parsing logic itself, not infrastructure, so there's nothing to "replace" with a managed service	N/A — this is a case where the right choice is *not* to look for an AWS service. A managed "SQL firewall" product would be a black box you can't unit-test the way you can test sqlglot's parse tree checks directly
12	Data warehouse	Snowflake, BigQuery, self-managed Postgres	Amazon Redshift Serverless (or Athena if data already lives in S3)	Serverless means no cluster sizing/pausing decisions, and it's the natural warehouse choice if billing/usage data is already in an AWS account	Over Snowflake: no cross-cloud data egress cost or separate billing relationship if the rest of the stack is AWS-native; over a fixed-size Redshift cluster: scales to zero when idle, so cost tracks actual query volume rather than provisioned capacity
13	Query cost estimation	Custom EXPLAIN-parsing logic	Redshift's/Athena's native `EXPLAIN` (Athena also exposes bytes-scanned estimates directly)	Athena in particular exposes a straightforward pre-execution bytes-scanned estimate, which	Over a fully custom cost model: the warehouse's own planner already computes this —

#	Component	Industry tool	AWS service	Why chosen	Advantage over alternatives
				maps directly onto the design doc's "EXPLAIN cost estimate under threshold" check without needing to parse a full query plan	using it directly is more accurate than reverse-engineering an independent cost estimate
14	Row-level security / DB access control	Postgres RLS policies, application-layer filtering	Redshift's native row-level security policies + IAM database authentication (temporary DB credentials, no static passwords)	IAM database authentication means the read-only role's credentials are short-lived and tied to the calling Lambda's IAM role — no static DB password to rotate or leak	Over static DB credentials + app-layer filtering: removes an entire class of credential-leak risk, and RLS enforcement happens in the database itself regardless of what the application layer does or fails to do

5. Data plane — retrieval path

#	Component	Industry tool	AWS service	Why chosen	Advantage over alternatives
15	Vector database	Pinecone, Weaviate, Milvus	Amazon OpenSearch Serverless (vector engine)	Pairs natively with Bedrock Knowledge Bases (below), so ingestion → indexing → retrieval is one managed path rather than a separate vendor integration	Over Pinecone: no separate vendor contract, API keys, or data-residency question — vectors stay inside the same AWS account/VPC boundary as everything else

#	Component	Industry tool	AWS service	Why chosen	Advantage over alternatives
16	Keyword (BM25) search	Elasticsearch, Apache Solr	Amazon OpenSearch Serverless (same cluster, keyword + vector in one engine)	OpenSearch natively supports both BM25 and k-NN vector search in the same index, which is exactly the hybrid retrieval pattern (Section 8.3 of the design doc) — one engine instead of two	Over running Elasticsearch + a separate vector DB: one system to operate, one place reciprocal rank fusion reads from, instead of merging results across two independently-managed services
17	RAG orchestration (chunk → embed → retrieve → generate)	LlamaIndex, custom LangChain RAG chain	Amazon Bedrock Knowledge Bases	Managed chunking, embedding, hybrid retrieval, and citation-attributed generation in one service — this is the single biggest scope-reduction opportunity in the whole rebuild	Over LlamaIndex/custom RAG: removes an entire category of infrastructure (embedding pipeline, vector store wiring, retrieval-fusion code) that would otherwise need to be built and maintained; citations come back structured, ready for the synthesizer's verification step
18	Reranking (cross-encoder)	Cohere Rerank, sentence-transformers cross-encoder	Amazon Bedrock's Rerank API (Cohere Rerank model, available natively through Bedrock)	Same underlying Cohere rerank model the design doc already selected — available through Bedrock	Over calling Cohere's API directly: one contract, one region/compliance boundary, and it composes with the rest of the Bedrock-based pipeline without crossing a network

#	Component	Industry tool	AWS service	Why chosen	Advantage over alternatives
				without a separate Cohere API contract	boundary to a third-party API

6. Synthesis and guardrails

#	Component	Industry tool	AWS service	Why chosen	Advantage over alternatives
19	Answer synthesis LLM	OpenAI GPT-4-class API, self-hosted Llama	Amazon Bedrock (Claude)	Matches the design doc's own model choice (Claude tiers) and gives the highest-quality tier for user-facing prose, exactly where the design doc says quality should be prioritized (Section 8.5, "model resilience")	Over calling Anthropic's API directly: Bedrock adds AWS IAM-based access control, VPC-private connectivity (no public internet egress required), and consolidated billing with the rest of the AWS spend

#	Component	Industry tool	AWS service	Why chosen	Advantage over alternatives
20	Guardrails (hallucination + PII)	Guardrails AI, NeMo Guardrails, custom regex/LLM-judge checks	Amazon Bedrock Guardrails	Built-in contextual grounding checks (verifies output is supported by the retrieved context — directly replaces the custom hallucination-checker LLM call) and native PII detection/redaction, configured declaratively rather than coded	Over Guardrails AI/NeMo: no separate library to keep in sync with model updates; grounding and PII checks run as a managed policy attached to the Bedrock call itself, so there's no custom post-processing step to maintain
21	Citation enforcement	Custom regex-based citation parser (as in the reference scaffold)	Bedrock Knowledge Bases' native citation/attribution output, combined with Bedrock Guardrails' grounding check	Knowledge Bases already returns which source chunks contributed to which parts of the answer — verifying against that native structure is more reliable than parsing free-form `[cite:id]` tags out of raw model output	Over custom regex parsing: removes an entire class of parsing-failure risk (malformed tags, inconsistent formatting) since the citation structure is a first-class part of the API response, not text the model has to format correctly

7. Shared services

#	Component	Industry tool	AWS service	Why chosen	Advantage over alternatives
22	Result cache	Redis (self-hosted or Redis Cloud)	Amazon ElastiCache for Redis (or ElastiCache Serverless)	Same Redis API the design doc already specifies, but with automated failover, patching, and scaling handled by AWS	Over self-hosted Redis: no cluster to patch or fail over manually; Serverless mode scales to actual traffic instead of a fixed node size sized for peak load
23	Immutable audit log	Kafka + ELK stack, Splunk	Amazon DynamoDB (write-only IAM policy, no update/delete grants) + AWS CloudTrail for infra-level audit	DynamoDB gives single-digit-millisecond writes at the query volume this system expects, and denying update/delete at the IAM policy level enforces immutability structurally, not by convention	Over Splunk: no per-GB ingestion licensing cost, and the immutability guarantee comes from IAM (a boundary AWS already provides) rather than from a separate access-control layer bolted onto the logging tool
24	Secrets management	HashiCorp Vault	AWS Secrets Manager	Native automatic rotation for RDS/Redshift credentials, and IAM-based access control means no separate Vault auth backend to configure	Over Vault: no separate service to run/secure/back up; rotation for AWS-native data stores (Redshift, RDS) is built in rather than requiring a custom rotation Lambda

#	Component	Industry tool	AWS service	Why chosen	Advantage over alternatives
25	Observability (infra tracing)	Prometheus + Grafana	Amazon CloudWatch + AWS X-Ray	X-Ray traces requests across Lambda, Step Functions, and API Gateway natively with no agent installation; CloudWatch consolidates logs/metrics from every AWS service in one place	Over Prometheus/Grafana: zero infrastructure to run for the metrics store itself — over half the value of Prometheus (service discovery, scraping) is moot when every component is already an AWS-managed service emitting metrics natively
26	Observability (LLM-specific tracing)	LangSmith, Weights & Biases Prompts	LangSmith (kept as-is — **no AWS-native equivalent exists yet for prompt/token-level tracing**)	LLM-specific concerns (prompt versions, token counts, per-call cost) aren't something CloudWatch models natively — this is a case for keeping the specialized tool rather than forcing an AWS substitute	This is the one row where the "AWS-native" instinct should be resisted — CloudWatch/X-Ray complement LangSmith for infra-level tracing, but replacing LangSmith outright would lose prompt-level visibility the design doc explicitly calls for (Section 8.5)

8. Async ingestion pipeline

#	Component	Industry tool	AWS service	Why chosen	Advantage over alternatives
27	Event bus	Apache Kafka (self-managed or Confluent)	Amazon Kinesis Data Streams (or Amazon MSK	Kinesis needs no cluster/broker management at all; MSK is the right	Over self-managed Kafka: no broker/Zookeeper cluster to run, patch, or scale —

#	Component	Industry tool	AWS service	Why chosen	Advantage over alternatives
			if you specifically need Kafka API compatibility)	choice only if you already have Kafka-specific tooling elsewhere that needs API compatibility	Kinesis scales shards automatically with throughput, and MSK Serverless offers a middle ground if Kafka API compatibility is required without managing brokers
28	Chunking + embedding compute	Custom Python service (Celery workers, etc.)	AWS Lambda (or Bedrock Knowledge Bases' automatic ingestion job if documents already sit in S3)	If using Bedrock Knowledge Bases, this step often disappears entirely — pointing it at an S3 prefix triggers automatic chunk/embed/index without any custom compute	Over custom worker pools: Lambda's per-invocation billing matches ingestion's bursty nature (new documents arrive irregularly), and Knowledge Bases' auto-sync removes this step as custom code altogether
29	Data catalog / lineage	OpenMetadata, DataHub	AWS Glue Data Catalog	Glue Catalog is the native AWS metadata store, and if Athena/Redshift Spectrum are already in the stack, tables are automatically discoverable through it	Over OpenMetadata/DataHub: no separate metadata service to run — Glue Catalog is already the metadata layer Athena and Redshift Spectrum use natively, so there's no second catalog to keep in sync
30	Raw document storage	Self-managed object store (MinIO)	Amazon S3	The default source-of-truth store for anything else in the pipeline (Bedrock Knowledge Bases, Athena, Glue Catalog) reads from S3 natively	Over MinIO or another self-hosted object store: every other AWS service in this list already integrates with S3 directly — versioning, lifecycle policies, and object lock (for immutability) are native features, not something to configure separately

Quick-reference: rows where "AWS-native" is genuinely the wrong call

Not every component should be replaced with a managed service. Two categories deserve a second look before defaulting to AWS:

- `sqlglot` **(row 11)** — this is application logic (AST parsing), not infrastructure. There's no managed service that gives you the same fine-grained, unit-testable control over the validation decision tree.
- **LangSmith (row 26)** — prompt/token-level LLM tracing is a genuinely specialized concern that CloudWatch doesn't model. Keeping a best-of-breed tool alongside AWS-native infra tracing is the right call here, not a compromise.

Everywhere else in this table, the AWS service either matches or exceeds the open-source/industry-standard tool's capability while removing an entire operational burden (patching, scaling, HA, credential rotation) — which is the actual argument for choosing it, not just "it's on AWS."